AVL Tree

An AVL tree is a self-balancing binary search tree in which each node has a balance factor that must lie in the range of −1, 0, or +1 to maintain its balancing property. This ensures that the tree remains height-balanced. It was introduced by G. M. Adelson-Velsky and E. M. Landis in 1962 and is named after them Adelson–Velsky and Landis tree.

Balance Factor (B.F.): The balance factor of a node in an AVL tree is defined as the difference between the heights of its left and right subtrees. Mathematically, it is calculated as:

B.F. = Height of Left Subtree-Height of Right Subtree = H_L-H_R

The balance factor helps maintain the self-balancing property of the AVL tree. For the tree to remain balanced, the balance factor of every node must be −1, 0, or +1. If the balance factor of any node falls outside this range, the tree becomes unbalanced at that node. In such cases, rotations (single or multiple) are performed to restore balance and maintain the AVL property.

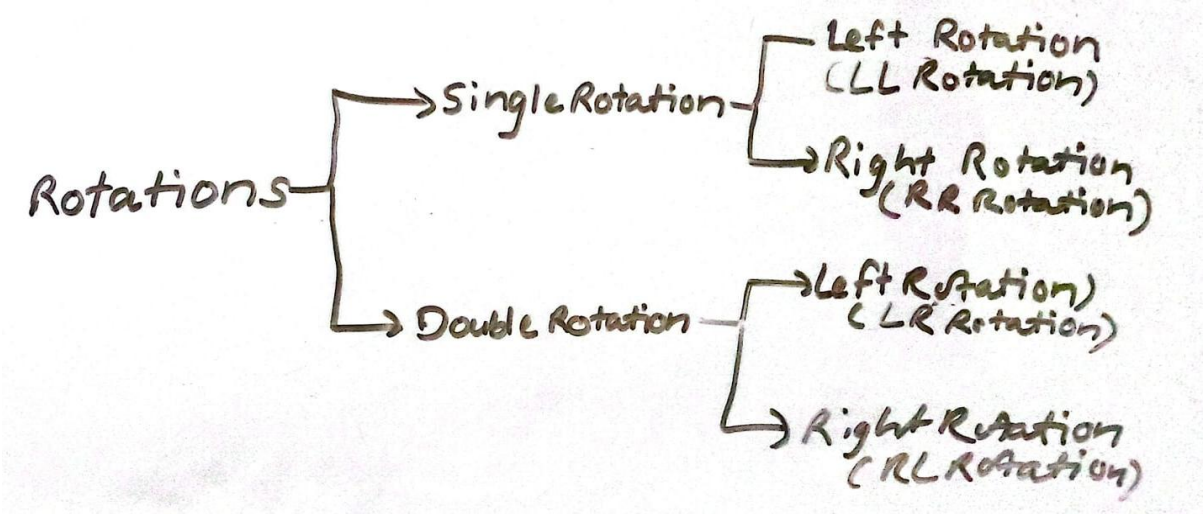
Rotations in AVL Tree:

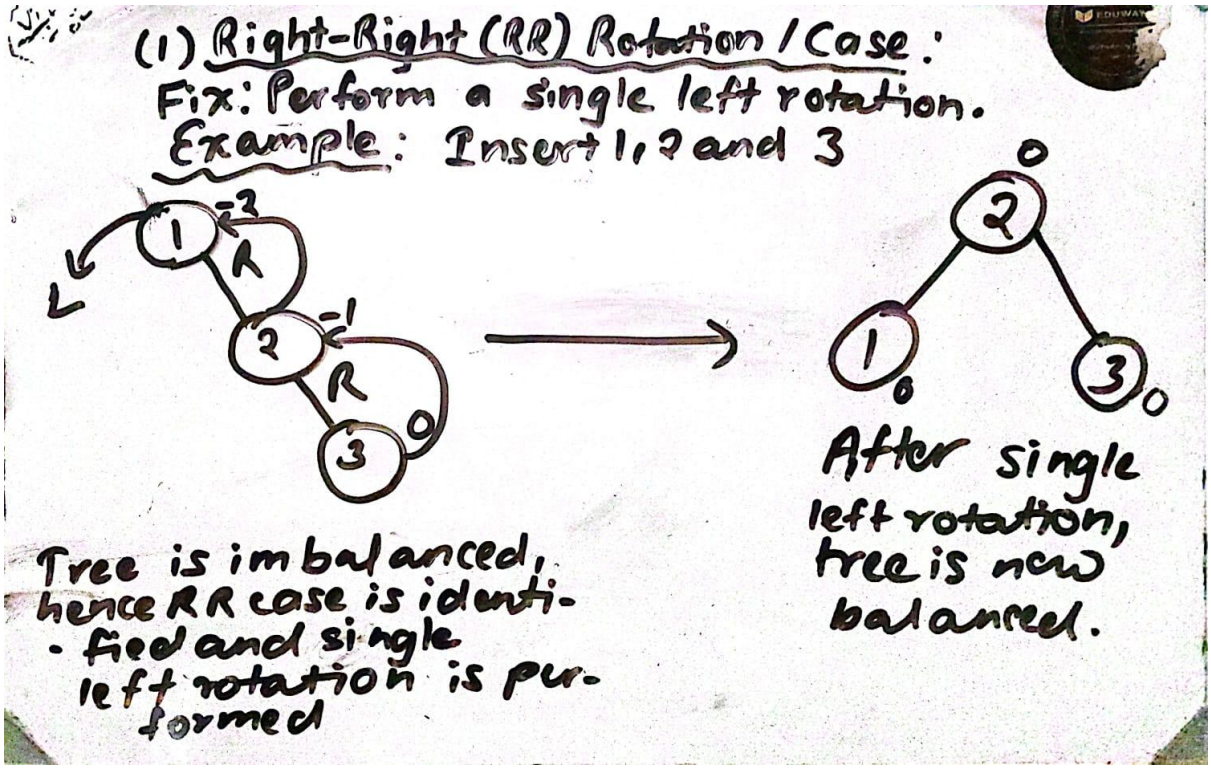
After performing operations such as insertion and deletion, it is necessary to check whether the binary search tree remains balanced. This is done by examining the balance factor of every node in the tree. If all nodes satisfy the balance factor condition of an AVL tree (−1, 0, or +1), the tree is considered balanced after the operation.

However, if any node does not satisfy this condition, the tree becomes unbalanced. In such cases, rotations are applied to restore balance. These rotations restructure the tree and convert the unbalanced tree back into a balanced one.

Thus, rotation is the process of moving nodes either to the left or to the right to maintain the balance of the tree.

There are four types of rotations based on different imbalance cases, which are classified into two main categories.

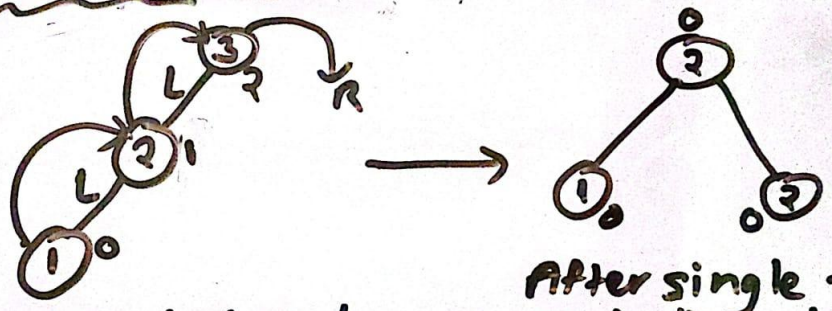




(1) Left-Left (LL) Rotation / Case:

Fix: Perform a single rt rotation.

Example: Insert 3, 2 & 1



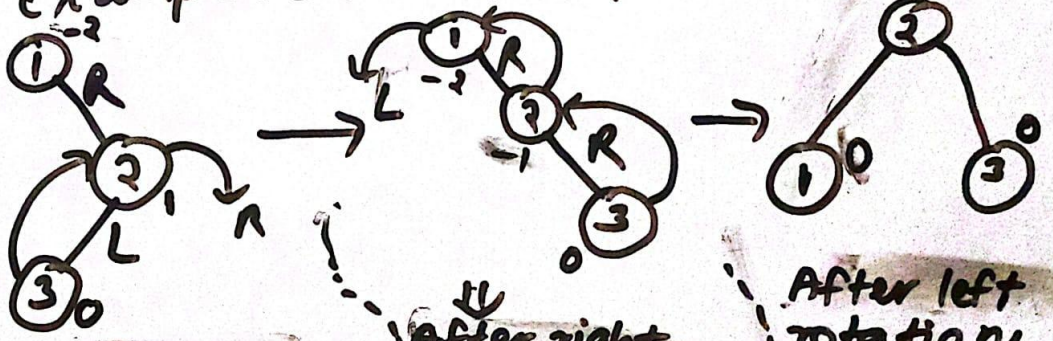
Tree is imbalanced, hence LL case is identified and single rt. rotation is performed.

After single rt. rotation, tree is now balanced.

(3) Right-Left (RL) Rotation / Case

Fix: **Perform first right rotation and then left rotation.**

Example: Insert 1, 3 & 2



Tree is imbalanced, hence RL case is identified and double rotation (right-left) is applied

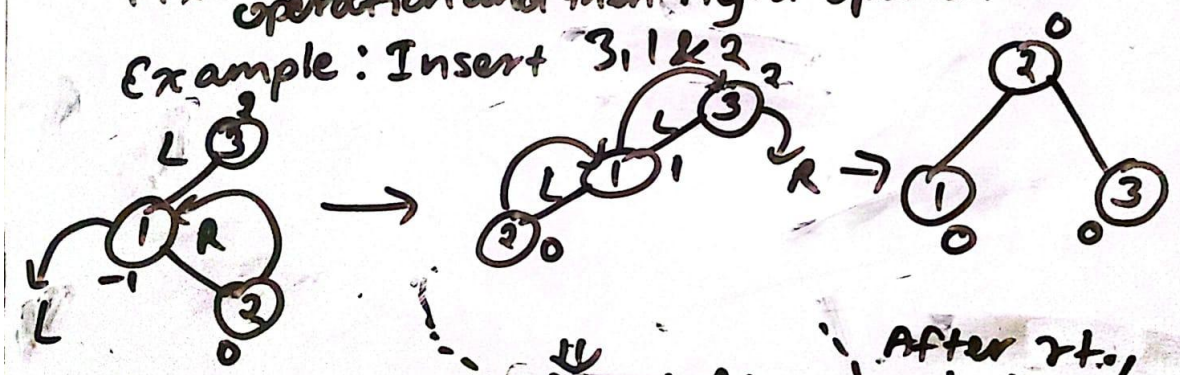
After right rotation, we use left rotation

After left rotation, tree is now balanced

(3) Left-Right (LR) Rotation/Case

Fix: Perform first perform left operation and then right operation

Example: Insert 3, 1 & 2



Tree is imbalanced, hence LR case is identified and double rotation (left-right) is applied

After left rotation, we use rt. rotation

After rt. rotation, tree is now balanced

Note: The balance factor (B.F.) of every node in the binary search tree must be calculated before and after insertion and deletion operations.

Operations in AVL Tree

The following operations are performed on an AVL tree:
- Searching
- Insertion
- Deletion

Insertion Operations in AVL Tree

Insertion in an AVL tree is similar to insertion in a regular binary search tree (BST), with the additional step of maintaining balance.
Steps for Deletion:

1. Insert elements one by one into the tree using the standard BST insertion logic. This constructs a BST.
2. After each insertion, check the balance factor of every node to ensure the AVL property is maintained. The balance factor (B.F.) of each node must be −1, 0, or +1.
3. If any node has a balance factor outside the range of −1, 0, or +1, it is called a critical node. This node causes the tree to become unbalanced, violating the AVL property. To restore balance, rotations are performed on the tree.
4. Repeat this process until the tree becomes balanced. The final balanced BST is called an AVL tree.

Key Concepts:

1. If the balance factor (B.F.) of a node is −1, 0, or +1, the node is balanced and not a critical node. However, if the B.F. becomes +2 or −2, it triggers a rotation to restore balance.
2. If multiple nodes become unbalanced (B.F. = ±2) after insertion, we must decide which node to fix first either the topmost, the

lowermost, or a random unbalanced node. In such ambiguous cases, we choose the lowermost unbalanced node

Why the lowest one?

- Because:
  - Imbalance starts from the bottom of the tree.
  - Fixing the lowest nodes automatically restores balance to upper nodes.
  -]If the topmost unbalanced node is rotated first, the tree may still remain unbalanced.

3. We choose the node whose balance factor is different (called the critical node) and perform rotations involving three nodes to restore balance.

Deletion operations in AVL tree

Deletion in an AVL tree is more complex due to the need to maintain the tree's balance.

Steps for Deletion:

1. **Delete Like a BST**: Remove the node as in a standard BST. There are three cases:
   a. **Node with No Child**: Simply remove the node.
   b. **Node with One Child**: Remove the node and replace it with its child.
   c. **Node with Two Children**: Find the in-order successor (smallest node in the right subtree) or in-order predecessor (largest in the left subtree), copy its value to the node, and delete the successor/predecessor.
2. **Check for Balance:** After deletion, walk back up the tree checking the balance factor of each node.
3. **Rotate if Necessary:** If any node is unbalanced (balance factor of -2 or 2), perform rotations as in the insertion case to balance the tree.

Searching operations in AVL tree

Searching in an AVL tree is identical to searching in a regular BST, as the AVL tree maintains the BST property.

Steps for Searching:

1. **Start at the Root:** Begin at the root node.

2. **Compare and Traverse:**
   a. Compare the search value with the current node's value.
   b. If it matches, the node is found.
   c. If the search value is smaller, move to the left child; if larger, move to the right child.
3. **Repeat or End:** Continue this process until the item is found or a leaf is reached.

Balanced Algorithm

A balanced algorithm, commonly referred to as a balanced tree algorithm, is a method that ensures the height difference between the left and right subtrees of any node is at most 1. This maintains high-performance search, insertion, and deletion operations by preventing the tree from becoming skewed (linear).

The main purpose of this algorithm is to avoid performance degradation in a binary search tree, keeping operations logarithmic (O(log n)) rather than linear (O(n)). This concept is used in AVL trees, B-trees, B+-trees, and red-black trees.

Time Complexity of AVL Tree:

Operation	Time Complexity
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)

Note:

AVL tree keeps height balanced (height = O(log n)), so all operations remain efficient. Rotations take constant time and do not affect overall complexity.